

Graph Databases in Rust: A Deep Dive

Scraping, Modelling, and Loading a Knowledge Graph with FalkorDB, HelixDB, and SurrealDB

A Study of the meetup-scraper Codebase

2026

Preface

This book uses a real, working Rust codebase — a Meetup event scraper and graph loader — as a vehicle for learning Rust. The codebase is compact enough to fit in your head but rich enough to demonstrate almost every feature you will reach for in production: `async/await`, traits with default methods, multiple error-handling strategies, lazy statics, procedural macros, type-parameterised SDKs, zero-copy string processing, and thorough integration testing without a live database.

By the end you will understand:

- How a real knowledge graph is modelled in JSON and then loaded into three different graph databases.
- How Rust’s module system, trait system, and type system work together to support pluggable back-ends without runtime cost.
- How `async fn` in traits, Semaphore-bounded concurrency, and `spawn_blocking` let you mix `async` and synchronous code cleanly.
- How the `serde`, `reqwest`, `clap`, `helix-db`, and `surrealdb` crates work in practice.
- Twelve concrete Rust language features and design patterns illustrated by the code.

All code excerpts in this book are taken verbatim from the repository. Nothing is invented or simplified.

Part I — The Domain

Chapter 1 — What the Codebase Does

The codebase solves a concrete data-engineering problem: extract structured information about tech meetup events from the Meetup.com website and load that information into graph databases so that questions like “*which speakers have co-presented?*” or “*what groups has this person spoken at?*” can be answered with graph traversals.

The information flow

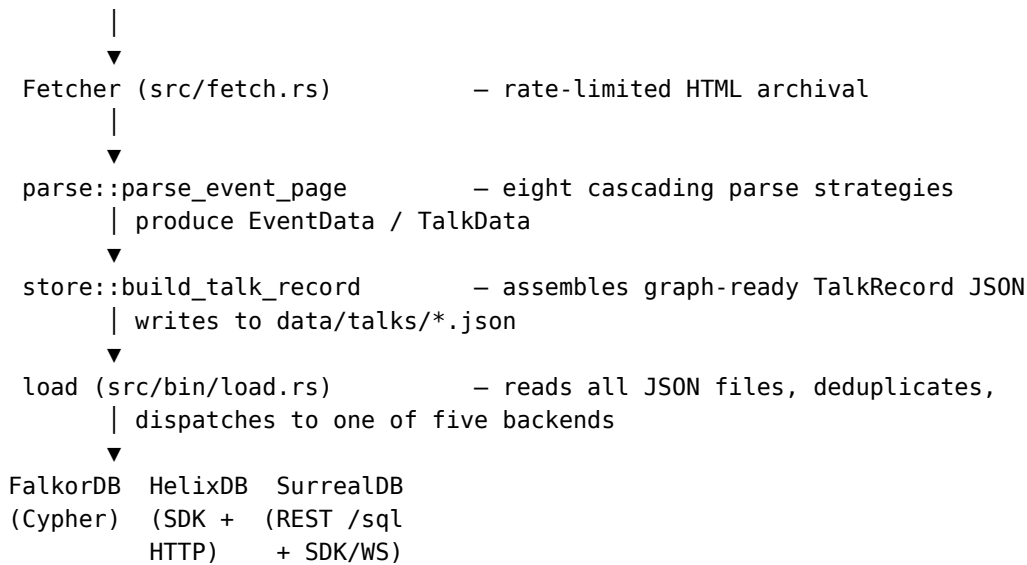
Meetup.com GraphQL API

|



GqlClient (src/gql.rs)

— paginated GraphQL requests



The scraper and loader are separate binaries. The scraper writes one JSON file per talk; the loader reads all of them, collapses duplicates, and performs a bulk upsert. This separation is deliberate: scraping is slow and rate-limited; loading should be fast and repeatable.

What is in the data?

After scraping eight Bay-Area tech meetup groups spanning 2012–2026, the dataset contains:

Entity	Count
Talks	163
Events	124
Groups	7
Speakers	181
PRESENTED_AT edges	163
PRESENTED_BY edges	215
PART_OF edges	124

The difference between 163 talks and 215 PRESENTED_BY edges shows that many talks have multiple co-speakers.

Chapter 2 — The Graph Model

Nodes and edges

The graph has four node labels and three edge types:

```

Talk —[PRESENTED_AT]→ Event —[PART_OF]→ Group
Talk —[PRESENTED_BY]→ Speaker

```

Every Talk is connected to exactly one Event; every Event is connected to exactly one Group; a Talk can have one or more Speakers.

The canonical node IDs

Node IDs follow a human-readable slug convention:

```
talk:bythebay-20120727-monad-transformers-in-scalamachine-and-scalia-jordan-west
event:sf-scala-69910422
group:sf-scala
speaker:jordan-west
```

The `store::slugify` function converts arbitrary text to a URL-safe slug: replace every non-alphanumeric character with a hyphen, collapse runs of hyphens, then trim to 60 characters.

```
pub fn slugify(text: &str) -> String {
    let parts: Vec<String> = text
        .to_lowercase()
        .chars()
        .map(|c| if c.is_ascii_alphanumeric() { c } else { '-' })
        .collect::<String>()
        .split('-')
        .filter(|s| !s.is_empty())
        .map(str::to_string)
        .collect();
    let slug = parts.join("-");
    if slug.len() > 60 {
        slug[..60].trim_end_matches('-').to_string()
    } else {
        slug
    }
}
```

The chain of iterator adaptors is idiomatic Rust:

1. `text.to_lowercase()` — produces a new `String`.
2. `.chars()` — produces a `Chars` iterator over Unicode scalar values.
3. `.map(|c| ...)` — replaces non-alphanumeric characters in place.
4. `.collect::<String>()` — reassembles into a `String` (Rust knows how to collect `char` items into a `String` via the `FromIterator<char> impl`).
5. `.split('-')` — splits into `&str` slices, which borrow the intermediate `String`.
6. `.filter(|s| !s.is_empty())` — drops empty segments created by consecutive hyphens.
7. A second `.map(str::to_string)` converts `&str` borrows to owned `Strings` so the final `Vec` does not borrow a temporary.
8. `.collect()` — infers `Vec<String>` from the variable declaration.

The TalkRecord JSON format

Each JSON file represents one graph subgraph centred on a single talk:

```
{
  "schema_version": "1.0",
```

```

    "id": "bythebay-20120727-monad-transformers-...-jordan-west",
    "scraped_at": "2026-05-10T08:34:24.501529Z",
    "source_file": "source/sf-scala/69910422.html",
    "nodes": [
      { "id": "talk:bythebay-...", "type": "Talk",
        "properties": { "title": "Monad Transformers ...", "order": 0 } },
      { "id": "event:sf-scala-69910422", "type": "Event",
        "properties": { "date": "2012-07-27", ... } },
      { "id": "group:sf-scala", "type": "Group",
        "properties": { "slug": "sf-scala", ... } },
      { "id": "speaker:jordan-west", "type": "Speaker",
        "properties": { "name": "Jordan West", ... } }
    ],
    "edges": [
      { "from": "talk:...", "to": "event:...", "type": "PRESENTED_AT" },
      { "from": "event:...", "to": "group:...", "type": "PART_OF" },
      { "from": "talk:...", "to": "speaker:...", "type": "PRESENTED_BY" }
    ]
  }
}

```

The loader reads every file in `data/talks/`, collapses node duplicates by ID, and loads each kind once. A speaker who appears in ten talk files will still produce only one Speaker node in the database.

Part II — Module Structure

Chapter 3 — The Library Modules

The main library crate (`src/`) is structured around the scraping pipeline:

```

src/
├─ main.rs           - top-level binary: orchestrates scraping
├─ config.rs         - constants (group slugs, schema version)
├─ gql.rs            - GraphQL client for Meetup's API
├─ fetch.rs          - rate-limited HTML fetcher with retry
├─ parse.rs          - eight cascading parsing strategies
├─ models.rs         - core data types (TalkRecord, Node, Edge, ...)
└─ store.rs          - slug/ID helpers and graph record builder

src/bin/
├─ load.rs           - the multi-backend graph loader (main focus of this book)
├─ helix_load.rs     - older HelixDB-specific loader (superseded by load.rs)
├─ helix_gen.rs      - generates HelixDB stored query JSON
├─ llm_extract.rs    - LLM-assisted event description extraction
└─ falkor_load.rs    - older FalkorDB-specific loader (superseded by load.rs)

```

Rust's module system ties these together cleanly. The library modules live under `src/` and are declared in `main.rs` with `mod` statements:

```
mod config;
mod fetch;
mod gql;
mod models;
mod parse;
mod store;
```

The `load.rs` binary is a separate compilation unit with its own `mod` declarations for the five backends. It does not share the library modules — it is self-contained, with its own `TalkRecord` deserialization type, `collect()` function, and all five backend implementations.

Inline modules vs. separate files

The five backends in `load.rs` are declared as *inline modules* using `mod backend_name { ... }` blocks within the same file. This keeps all related loader code in one place and makes it easy to read the full backend in one scroll. Each inline module imports only what it needs from the parent module with `use super::*` or explicit `use super::XYZ`.

Contrast this with the library modules, which are in *separate files*. Both are syntactically identical to Rust — the distinction is purely organizational.

Chapter 4 — The `models.rs` Structs

```
#[derive(Debug, Clone, Serialize, Deserialize)]
pub struct TalkRecord {
    pub schema_version: String,
    pub id: String,
    pub scraped_at: DateTime<Utc>,
    pub source_file: String,
    pub nodes: Vec<Node>,
    pub edges: Vec<Edge>,
}

#[derive(Debug, Clone, Serialize, Deserialize)]
pub struct Node {
    pub id: String,
    #[serde(rename = "type")]
    pub kind: String,
    pub properties: serde_json::Value,
}

#[derive(Debug, Clone, Serialize, Deserialize)]
pub struct Edge {
    pub from: String,
    pub to: String,
    #[serde(rename = "type")]
    pub kind: String,
    #[serde(skip_serializing_if = "Option::is_none")]
    pub properties: Option<serde_json::Value>,
}
```

Language features on display

Derive macros — `#[derive(Debug, Clone, Serialize, Deserialize)]` automatically generates trait implementations at compile time. `Debug` allows `{:?}` formatting; `Clone` allows `.clone()`; `Serialize` and `Deserialize` come from `serde` and handle JSON I/O.

serde(rename) — The JSON field is called "type" but `type` is a reserved keyword in Rust, so the struct field is called `kind`. The `#[serde(rename = "type")]` attribute bridges the gap: `serde` reads and writes "type" in JSON while Rust code uses `kind`.

serde(skip_serializing_if) — Edge properties are often absent. The attribute `#[serde(skip_serializing_if = "Option::is_none")]` tells `serde` to omit the "properties" key entirely when the value is `None`, producing cleaner JSON. The string `"Option::is_none"` is a *path* to a function with signature `fn(&T) -> bool`.

serde_json::Value — The `properties` field uses the dynamic `serde_json::Value` type instead of a typed struct because different node kinds have different properties. This trades compile-time safety for flexibility. Later chapters will show how `node_props()` enforces structure at the point of database insertion.

Part III — Data Collection

Chapter 5 — The GraphQL Client (`gql.rs`)

Meetup.com exposes a GraphQL API at <https://www.meetup.com/gql2>. The client is built around `request` and paginated cursor-based queries.

```
const EVENTS_QUERY: &str = r#"
query($urlname: String!, $after: String) {
  groupByUrlname(urlname: $urlname) {
    events(
      filter: { status: [ACTIVE, PAST, CANCELLED] }
      first: 20
      sort: DESC
      after: $after
    ) {
      totalCount
      edges { node { id title dateTime description eventUrl venue { name address
city } } }
      pageInfo { hasNextPage endCursor }
    }
  }
}
"#;
```

The raw string literal `r#"..."#` avoids the need to escape the embedded double quotes. The `#` delimiters can be repeated to nest `"#` inside the string.

Pagination loop

```
pub async fn all_events(&self, group: &str) -> Vec<GqlEvent> {
    let mut all: Vec<GqlEvent> = Vec::new();
    let mut cursor: Option<String> = None;

    loop {
        sleep(self.delay).await;
        match self.fetch_page(group, cursor.as_deref()).await {
            Ok((events, _total, next_cursor, has_next)) => {
                all.extend(events);
                if has_next { cursor = Some(next_cursor); } else { break; }
            }
            Err(e) => { warn!(...); break; }
        }
    }
    all
}
```

`cursor.as_deref()` converts `Option<String>` to `Option<&str>` — it calls `as_deref` which applies `Deref (String → &str)` inside the `Option`. This is a common idiom that avoids cloning just to pass a short-lived string reference.

Deserializing the response

```
let events: Vec<GqlEvent> = edges
    .iter()
    .filter_map(|e| {
        serde_json::from_value::<GqlEvent>(e["node"].clone())
            .map_err(|err| { warn!("deserialize event: {err}"); err })
            .ok()
    })
    .collect();
```

`filter_map` is used instead of `map + flatten` to skip events that fail to deserialize. The `map_err` logs the error before `.ok()` discards it, combining logging and error conversion in one chain without extra variables.

Chapter 6 — The HTML Fetcher (`fetch.rs`)

The fetcher must be polite: one request at a time, a configurable delay between them, and exponential back-off on failure.

```
pub struct Fetcher {
    client: request::Client,
    delay: Duration,
    sem: Semaphore,      // permits = 1 → sequential
    max_retries: u32,
}
```

A Semaphore with a single permit enforces sequential access even if the fetcher were to be called from multiple async tasks. The critical section is the entire `get()` method body:

```
pub async fn get(&self, url: &str) -> Result<String> {
    let _permit = self.sem.acquire().await?;
    sleep(self.delay).await;
    // ... retry loop ...
}
```

`_permit` is a guard value: its lifetime controls the critical section. When `_permit` drops at the end of the function the permit is returned to the semaphore. The leading underscore is a Rust convention that says “*I intentionally hold this value for its drop behaviour but never read it.*”

Exponential back-off

```
for attempt in 0..self.max_retries {
    if attempt > 0 {
        let backoff = self.delay * 2u32.pow(attempt);
        warn!("Retry {attempt} for {url}, backoff {}ms", backoff.as_millis());
        sleep(backoff).await;
    }
    // ... send request ...
}
```

`2u32.pow(attempt)` computes 2, 4, 8 on retries 1, 2, 3. Rust’s numeric literals include an optional type suffix — `2u32` is a `u32` value 2. `Duration * u32` is implemented by the standard library and produces another `Duration`.

HTTP status handling

```
if status.as_u16() == 429 {
    warn!("Rate-limited on {url}, waiting 60 s");
    sleep(Duration::from_secs(60)).await;
    continue;
}
if status.as_u16() == 403 {
    return Err(anyhow!(
        "403 Forbidden for {url}. Export your browser cookies..."
    ));
}
```

403 is a hard failure — Meetup.com requires authentication and there is no point retrying. 429 is a soft failure — wait and try again. The `continue` statement restarts the retry loop without consuming a retry count.

Chapter 7 — The Parser (`parse.rs`)

The parser is the most complex module. Meetup event descriptions are free-form Markdown text. Different meetup groups use wildly different formatting conventions, so the parser implements *eight cascading strategies*, each specialised for one style.

The `try_strategy!` macro

```
macro_rules! try_strategy {
    ($fn:expr) => {{
        let talks = $fn(&md);
        if talks.iter().any(|t| !t.speakers.is_empty()) {
            return renumber(
                talks.into_iter().filter(|t| !t.speakers.is_empty()).collect()
            );
        }
    }};
}
```

The macro evaluates the strategy function and returns immediately if it produces any talks with speakers. The double braces `{{ }}` inside the macro rule create a block expression so that `return` exits the enclosing function, not just the macro expansion. Each strategy is tried in order:

```
try_strategy!(talks_bold_title_speaker); // **Title** / ***Speaker*, Company**
try_strategy!(talks_numbered_sections); // (1) Title then prose
try_strategy!(talks_explicit_labels); // Presentation: / Speaker:
try_strategy!(talks_md_header_speaker); // # Title then **Speaker:** Name
try_strategy!(talks_quoted_title); // "Title" then Name, Company
try_strategy!(talks_bullet_period); // • Title. Name, Company
try_strategy!(talks_bullet_dash); // * **Title** - *Speaker*
try_strategy!(talks_time_slots); // 6:35 - Title - Name
```

OnceLock for lazy regex compilation

Regular expressions are expensive to compile but cheap to evaluate. The pattern below compiles each regex exactly once, on first use:

```
fn re_bold_title_line() -> &'static Regex {
    static R: OnceLock<Regex> = OnceLock::new();
    R.get_or_init(|| Regex::new(r"^\[s\u{200B}\u{FEFF}]*\s*([^\n]
{4,}?)\s*\s*$").unwrap())
}
```

`OnceLock<T>` is a cell that can be written exactly once and then read forever without synchronization overhead. It lives in a function-local static so it has the lifetime `'static` — it is never freed. The return type `&'static Regex` is a reference to a value that lives forever.

`get_or_init` takes a closure that is called only once; subsequent calls return the already-computed value. The `unwrap()` on `Regex::new` is safe here because the pattern is a string literal that was validated at development time. If the pattern were invalid the binary would panic immediately on first use — a fail-fast approach that is acceptable for hard-coded patterns.

The speaker parsing heuristic

`parse_speaker_string` demonstrates Rust's power for multi-step string processing:

```
fn parse_speaker_string(raw: &str) -> Vec<SpeakerData> {
    let raw = raw.trim()
        .trim_start_matches("Speaker:");
```

```

        .trim_start_matches("Presenter:")
        // ...
        .trim();

// Split on "and" or "&" but only if the conjunction precedes any comma.
let comma_pos = raw.find(',');
let split_and = match (comma_pos, raw.find(" and ")) {
    (Some(c), Some(a)) => a < c,
    (None, _) => true,
    _ => true,
};

let stage1: Vec<&str> = if split_and {
    raw.split(" and ").collect()
} else {
    vec![raw]
};

let stage2: Vec<&str> = stage1.into_iter().flat_map(|s|
s.split(';')).collect();
// ... stage3 splits on " & " ...

stage3.into_iter()
    .filter_map(|p| {
        let p = p.trim();
        let (name, company) = split_name_company(p);
        // ... validate name shape ...
        Some(SpeakerData { name, company, bio: None })
    })
    .collect()
}

```

The key insight is the comma-before-conjunction check. The string "Name, Role with Company and Partner" should *not* be split on " and " because the conjunction is within the role description, not separating two speakers. Only if " and " appears *before* the first comma is it treated as a speaker separator.

Part IV — The Graph Loader

Chapter 8 — The `load.rs` Binary Architecture

`src/bin/load.rs` is the centrepiece of this book. It implements five pluggable database backends behind a single trait, demonstrates several important Rust patterns, and includes twelve unit tests that require no live database.

CLI with `clap` derive macros

```

#[derive(clap::ValueEnum, Debug, Clone)]
enum Backend {
    Falkor,

```

```

    #[value(name = "helix-http")]
    HelixHttp,
    #[value(name = "helix-sdk")]
    HelixSdk,
    #[value(name = "surreal-http")]
    SurrealHttp,
    #[value(name = "surreal-sdk")]
    SurrealSdk,
}

#[derive(Parser, Debug)]
#[command(about = "Load scraped talk JSON files into FalkorDB, HelixDB, or SurrealDB")]
struct Args {
    #[arg(short, long, default_value = "data/talks")]
    input: PathBuf,
    #[arg(long, value_enum, default_value_t = Backend::Falkor)]
    backend: Backend,
    #[arg(long)]
    url: Option<String>,
    #[arg(long, default_value_t = 100)]
    batch_size: usize,
    #[arg(long, env = "HELIXDB_API_KEY")]
    api_key: Option<String>,
    // ...
}

```

The `#[derive(Parser)]` macro from `clap` generates a full argument parser with `--help`, error messages, and shell completion support. Notice:

- `#[value(name = "helix-http")]` — renames an enum variant for the CLI, so the user types `--backend helix-http` rather than `--backend HelixHttp`.
- `default_value_t` — uses the `Display` trait to convert the Rust value to a string for the help text.
- `env = "HELIXDB_API_KEY"` — reads the value from an environment variable as a fallback, which is essential for CI/CD.

Graph collection and deduplication

```

type NodesByKind = BTreeMap<String, NodeKindMap>;
pub type NodeKindMap = BTreeMap<String, serde_json::Value>;
pub type EdgeKindMap = BTreeMap<(String, String), Option<serde_json::Value>>;

```

The `collect()` function reads every JSON file and builds these maps:

```

for node in rec.nodes {
    nodes
        .entry(node.kind)
        .or_default()
        .entry(node.id)
        .or_insert(node.properties);
}

```

BTreeMap::entry returns an Entry enum — either Occupied (key exists) or Vacant (key absent). or_default() inserts a default value (BTreeMap::default() is an empty map) if the key is absent and returns a mutable reference in both cases. or_insert similarly inserts only if absent, preserving the first-seen properties when the same node appears in multiple talk files.

Why BTreeMap instead of HashMap? The BTreeMap keeps keys in sorted order, which makes node and edge insertion deterministic across runs. When debugging a loading problem you get the same order every time, and tests that check insertion order are stable. The trade-off is $O(\log n)$ instead of $O(1)$ access, but with at most ~500 nodes this is imperceptible.

The node_props single source of truth

Before this function existed, each of the five backends maintained its own build_node_body() or build_node_content() function with identical logic. That is a classic maintenance hazard: a renamed field in one place silently leaves other backends stale. The solution is a single canonical function:

```
pub fn node_props(kind: &str, nid: &str, props: &serde_json::Value) ->
serde_json::Value {
    match kind {
        "Talk" => serde_json::json!({
            "nid":          nid,
            "title":        str_prop(props, "title"),
            "abstract_text": str_prop(props, "abstract"), // ← renamed
            "talk_order":   i64_prop(props, "order"),     // ← renamed
        }),
        "Event" => serde_json::json!({
            "nid": nid, "event_id": str_prop(props, "id"),
            "title": str_prop(props, "title"),
            // ... 6 more fields ...
        }),
        "Group" => serde_json::json!({ "nid": nid, "slug": ..., "name": ..., "url":
... }),
        _ => serde_json::json!({ "nid": nid, "name": ..., "bio": ..., "company":
... }),
    }
}
```

Notice the renames: the raw JSON from store.rs uses "abstract" and "order" but the graph databases receive "abstract_text" and "talk_order". These renames exist because abstract is a reserved keyword in some query languages and order is ambiguous. Having them in one place makes the intent clear.

The helper functions str_prop and i64_prop centralise default handling:

```
fn str_prop<'a>(props: &'a serde_json::Value, key: &str) -> &'a str {
    props.get(key).and_then(|v| v.as_str()).unwrap_or("")
}
fn i64_prop(props: &serde_json::Value, key: &str) -> i64 {
```

```

        props.get(key).and_then(|v| v.as_i64()).unwrap_or(0)
    }

```

`str_prop` uses a *lifetime annotation* `'a` to tell the compiler that the returned `&str` borrows from the same `props` value passed in. This avoids an allocation: the `&str` points directly into the `serde_json::Value`'s internal `String` storage.

Chapter 9 — The GraphLoader Trait

```

trait GraphLoader {
    /// Optional one-time setup. Default implementation is a no-op.
    async fn bootstrap(&self) -> Result<()> {
        Ok(())
    }

    async fn clear(&self) -> Result<()>;

    async fn load_nodes_batch(&self, kind: &str, nodes: &NodeKindMap) ->
    Result<BatchStats>;

    async fn load_edges_batch(
        &self,
        kind: &str,
        edges: &EdgeKindMap,
        from_label: &str,
        to_label: &str,
    ) -> Result<BatchStats>;
}

```

Async methods in traits

`async fn` in traits became stable in Rust 1.75 (December 2023). Before that, trait methods returning futures required the `async-trait` crate, which rewrites `async fn` to return `Box<dyn Future<...>>` — a heap allocation per call. Stable `async` traits are destigmatized and do not allocate.

Default method implementations

The `bootstrap` method has a default body `{ Ok(()) }`. Backends that do not need initialization (FalkorDB and HelixDB) inherit this no-op for free. Only `surreal_http` and `surreal_sdk` override it to create the namespace and database:

```

// In surreal_http::Loader:
async fn bootstrap(&self) -> Result<()> {
    // Step 1 — root level: DEFINE NAMESPACE
    if let Err(e) = self.run_sql(
        &format!("DEFINE NAMESPACE IF NOT EXISTS {}; ", self.ns),
        false, // ← no Surreal-Ns/Db headers at root level
    ).await {
        tracing::warn!("SurrealDB bootstrap (DEFINE NAMESPACE): {e}");
    }
    // Step 2 — within namespace: DEFINE DATABASE
}

```

```

    if let Err(e) = self.run_sql(
        &format!("DEFINE DATABASE IF NOT EXISTS {};", self.db),
        true,
    ).await {
        tracing::warn!("SurrealDB bootstrap (DEFINE DATABASE): {e}");
    }
    Ok(())
}

```

Notice that bootstrap errors are *logged but not propagated*. This is intentional: if the namespace already exists, `DEFINE NAMESPACE` returns an error even with `IF NOT EXISTS` in some versions. The loader treats bootstrap as best-effort; the real test is whether subsequent upserts succeed.

The generic dispatch function

```

async fn load_graph(
    loader: &impl GraphLoader,
    nodes: &NodesByKind,
    edges: &EdgesByKind,
    clear: bool,
) -> Result<()> {
    loader.bootstrap().await?;
    if clear { loader.clear().await?; }

    for &kind in NODE_KINDS {
        if let Some(n) = nodes.get(kind) {
            let s = loader.load_nodes_batch(kind, n).await?;
            info!("  {:>8}: {} loaded, {} skipped", kind, s.loaded, s.skipped);
        }
    }
    for (&(kind, from_label, to_label) in EDGE_SCHEMA {
        if let Some(e) = edges.get(kind) {
            let s = loader.load_edges_batch(kind, e, from_label,
to_label).await?;
        }
    }
    Ok(())
}

```

`&impl GraphLoader` is *impl Trait* syntax: the compiler monomorphizes `load_graph` for each concrete loader type, generating optimized code for each backend without virtual dispatch. The caller is:

```

match args.backend {
    Backend::Falkor => {
        let loader = falkor::FalkorLoader::new(&url, &args.graph)?;
        load_graph(&loader, &nodes, &edges, args.clear).await?;
    }
    Backend::HelixSdk => {
        let loader = helix_sdk::Loader::new(
            &url, args.api_key.as_deref(), args.concurrency, args.batch_size)?;
    }
}

```

```

        load_graph(&loader, &nodes, &edges, args.clear).await?;
    }
    // ...
}

```

The schema constants drive the loop:

```

const NODE_KINDS: &[&str] = &["Talk", "Event", "Group", "Speaker"];

const EDGE_SCHEMA: &[(&str, &str, &str)] = &[
    ("PRESENTED_AT", "Talk", "Event"),
    ("PRESENTED_BY", "Talk", "Speaker"),
    ("PART_OF", "Event", "Group"),
];

```

Node ordering matters: Talk nodes must exist before PRESENTED_AT edges can reference them. The constants are manually ordered to satisfy this dependency.

Part V — Backend Deep Dives

Chapter 10 — FalkorDB via Redis + Cypher

FalkorDB is a graph database built on top of Redis. It exposes graph queries through a Redis command: GRAPH.QUERY <graph_name> <cypher>. The Rust redis crate provides the transport.

The synchronous problem in an async world

The redis crate is entirely synchronous. Its connection object is not Send between async tasks and its API uses blocking I/O. Running blocking code inside a tokio async runtime stalls the thread pool. The solution is tokio::task::block_in_place:

```

async fn load_nodes_batch(&self, kind: &str, nodes: &NodeKindMap) ->
Result<BatchStats> {
    tokio::task::block_in_place(|| {
        let _ = self.cypher_query(&format!("CREATE INDEX FOR (n:{kind}) ON
(n.nid)"));
        let array = nodes_array(nodes);
        self.cypher_query(&format!(
            "UNWIND {array} AS row \
            MERGE (n:{kind} {{nid: row.nid}}) \
            SET n += row.props"
        ))
        .with_context(|| format!("MERGE {kind} nodes"))?;
        Ok(BatchStats { loaded: nodes.len(), skipped: 0 })
    })
}

```

block_in_place tells tokio “I am about to run blocking code; move other tasks to different threads while I am running.” It is the correct call when you own the blocking call site. The alternative,

`spawn_blocking`, moves the closure to a dedicated thread pool — useful when you need the blocking code to run concurrently with other async tasks.

Cypher UNWIND batching

Instead of sending one MERGE statement per node, the entire `NodeKindMap` is serialized into a single Cypher array literal and processed with UNWIND:

```
UNWIND [{nid: 'talk:foo', props: {title: 'My Talk', abstract_text: '...'}}, ...] AS row
MERGE (n:Talk {nid: row.nid})
SET n += row.props
```

This sends *one* network round-trip per node kind regardless of how many nodes there are. For 163 talks that is 1 round-trip instead of 163.

Building the Cypher literal

```
pub(super) fn nodes_array(nodes: &NodeKindMap) -> String {
    let items: Vec<String> = nodes
        .iter()
        .map(|(nid, props)| {
            format!("{{nid:{},props:{}}}", cypher_str(nid),
                json_to_cypher(props))
        })
        .collect();
    format!("[{}]", items.join(","))
}
```

The `pub(super)` visibility makes this function visible to the parent module (and thus to the unit tests at the bottom of `load.rs`) but not to external callers. This is more restrictive than `pub` but more permissive than the default (private to the current module).

String escaping

```
pub(super) fn cypher_str(s: &str) -> String {
    let mut out = String::with_capacity(s.len() + 2);
    out.push('\\');
    for c in s.chars() {
        match c {
            '\\' => out.push_str("\\\\"),
            '\'' => out.push_str("\\'"),
            '\n' => out.push_str("\\n"),
            '\r' => out.push_str("\\r"),
            '\t' => out.push_str("\\t"),
            c    => out.push(c),
        }
    }
    out.push('\\');
    out
}
```

`String::with_capacity(s.len() + 2)` pre-allocates enough space for the two surrounding quotes plus the content. Strings that need escaping will trigger reallocations but strings without

special characters will not. This is the correct balance: optimise the common case, allow the rare case to work correctly.

Chapter 11 — HelixDB via Raw HTTP (Stored Queries)

HelixDB is a purpose-built property graph database with a Rust SDK. The `helix-http` backend uses pre-generated *stored queries* — named query endpoints registered with the server at startup.

```
pub struct Loader {
    client: request::Client,
    base_url: Arc<String>,
    concurrency: usize,
}
```

`Arc<String>` is an *atomically reference-counted* smart pointer. The `base_url` is shared across many concurrently spawned async tasks; `Arc` allows this without copying the string. Each task clones the `Arc` (which is a cheap pointer copy) rather than the `String` content.

Concurrent task dispatch

```
async fn load_nodes_batch(&self, kind: &str, nodes: &NodeKindMap) ->
Result<BatchStats> {
    let ep = Arc::new(node_endpoint(kind).to_string());
    let sem = Arc::new(Semaphore::new(self.concurrency));
    let mut handles = Vec::new();

    for (nid, props) in nodes {
        let body = node_props(kind, nid, props);
        let client = self.client.clone();
        let base_url = self.base_url.clone();
        let ep = ep.clone();
        let sem = sem.clone();
        handles.push(tokio::spawn(async move {
            let _permit = sem.acquire_owned().await?;
            http_post(&client, &base_url, &ep, body).await
        }));
    }
    Ok(collect_concurrent(handles).await)
}
```

The semaphore limits how many HTTP requests are in-flight simultaneously. `acquire_owned()` returns a permit that is tied to the `Arc<Semaphore>` rather than the semaphore itself, allowing it to be moved into the async closure (closures that move data need `move` and the data must be `'static`; permit-by-reference would require the semaphore to outlive the closure, which is harder to express).

The `collect_concurrent` drain helper

```
async fn collect_concurrent(handles: Vec<tokio::task::JoinHandle<Result<()>>>) ->
BatchStats {
    let mut loaded = 0usize;
```

```

    let mut skipped = 0usize;
    for h in handles {
        match h.await {
            Ok(Ok(())) => loaded += 1,
            Ok(Err(e)) => { tracing::warn!(" task error: {e:#}"); skipped +=
1; }
            Err(e)      => { tracing::warn!(" join error: {e}"); skipped += 1; }
        }
    }
    BatchStats { loaded, skipped }
}

```

`h.await` can fail in two different ways:

- `Err(JoinError)` — the task *panicked* or was *cancelled*. This is the outer error.
- `Ok(Err( anyhow::Error ))` — the task ran to completion but returned an error. This is the inner error.

Both cases are counted as skipped. The double-match pattern `Ok(Ok(...)) / Ok(Err(...)) / Err(...)` handles both failure modes explicitly.

URL normalization

```

pub fn helix_base_url(raw: &str) -> String {
    let s = raw.trim_end_matches('/');
    let s = s.strip_suffix("/v1/query").unwrap_or(s);
    let s = s.trim_end_matches('/');
    s.to_string()
}

```

The function chains `str` method calls without allocating intermediate `Strings`. `trim_end_matches`, `strip_suffix`, and `trim_end_matches` all return `&str` slices into the original string. The allocation happens only at the end with `s.to_string()`. This is the zero-copy philosophy applied to string normalization.

Chapter 12 — HelixDB via the Rust SDK (Dynamic Batch Queries)

The `helix-sdk` backend uses the `helix-db` crate's DSL to build graph queries in Rust code rather than as stored query endpoints. This is the most Rust-idiomatic backend because the query shape is expressed as typed Rust values, not strings.

The client

```

pub struct Loader {
    client: Arc<Client>,
    concurrency: usize,
    batch_size: usize,
}

impl Loader {
    pub fn new(raw_url: &str, api_key: Option<&str>, concurrency: usize,

```

```

batch_size: usize) -> Result<Self> {
    let url = helix_base_url(raw_url);
    let client = Client::new(Some(&url))
        .map_err(|e| anyhow::anyhow!("HelixDB client: {e}"))?
        .with_api_key(api_key);
    Ok(Self { client: Arc::new(client), concurrency, batch_size:
batch_size.max(1) })
    }
}

```

Client::new returns a type that is not anyhow::Error, so map_err(|e| anyhow::anyhow!("{}", e)) wraps it. anyhow::anyhow! is a macro that creates an anyhow::Error from anything that implements Display.

Batching nodes into a single write_batch request

HelixDB conflicts when multiple concurrent writes touch the same region of the graph. The solution is to pack many node upserts into one write_batch() call. This reduces concurrent requests and groups related writes atomically:

```

fn make_node_batch(kind: &str, chunk: &[(String, serde_json::Value)]) ->
DynamicQueryRequest {
    let mut batch = write_batch();
    let mut names: Vec<String> = Vec::with_capacity(chunk.len());

    for (i, (nid, raw_props)) in chunk.iter().enumerate() {
        let name = format!("n{i}");
        let np = node_props(kind, nid, raw_props);
        let pairs = json_to_helix_pairs(&np);
        batch = batch.var_as(&name, g().add_n(kind, pairs));
        names.push(name);
    }
    DynamicQueryRequest::write(batch.returning(names.iter().map(|s| s.as_str())))
}

```

write_batch() returns a builder. Each var_as(name, traversal) call adds a named step and returns a new builder (builder pattern). The traversal g().add_n(kind, pairs) means “start at the graph root, add a node with label kind and properties pairs”. At the end, returning specifies which variables to include in the response.

Type conversion: JSON → PropertyValue

HelixDB’s SDK does not accept serde_json::Value; it requires its own PropertyValue enum:

```

fn json_to_helix_pairs(v: &serde_json::Value) -> Vec<(String, PropertyValue)> {
    let obj = match v { serde_json::Value::Object(m) => m, _ => return vec![] };
    obj.iter().map(|(k, v)| {
        let pv = match v {
            serde_json::Value::String(s) => PropertyValue::from(s.as_str()),
            serde_json::Value::Number(n) => {
                if let Some(i) = n.as_i64() { PropertyValue::I64(i) }
            }
        }
    })
}

```

```

        else if let Some(f) = n.as_f64() { PropertyValue::F64(f) }
        else { PropertyValue::Null }
    }
    serde_json::Value::Bool(b) => PropertyValue::Bool(*b),
    serde_json::Value::Null    => PropertyValue::Null,
    other => PropertyValue::from(other.to_string()),
};
(k.clone(), pv)
}).collect()
}

```

The nested match mirrors the JSON type hierarchy. Numbers are special: `serde_json::Number` does not commit to a specific numeric type, so the code checks for `i64` first (the common case for integer properties) and falls back to `f64`.

Edge batching with named variables

```

fn make_edge_batch(kind: &str, from_label: &str, to_label: &str,
    chunk: &[(String, String), Option<serde_json::Value>]) ->
DynamicQueryRequest {
    let mut batch = write_batch();
    let mut e_names: Vec<String> = Vec::with_capacity(chunk.len());

    for (i, ((from_nid, to_nid), _)) in chunk.iter().enumerate() {
        let src_var = format!("src{i}");
        let dst_var = format!("dst{i}");
        let e_var   = format!("e{i}");
        batch = batch
            .var_as(&src_var,
                g().n_with_label_where(from_label, SourcePredicate::eq("nid",
from_nid.as_str()))
                .limit(1))
            .var_as(&dst_var,
                g().n_with_label_where(to_label, SourcePredicate::eq("nid",
to_nid.as_str()))
                .limit(1))
            .var_as(&e_var,
                g().n(NodeRef::var(&src_var))
                .add_e(kind, NodeRef::var(&dst_var), Vec:::<(&str,
&str)>::new())
                .count());
        e_names.push(e_var);
    }
    DynamicQueryRequest::write(batch.returning(e_names.iter().map(|s|
s.as_str()))))
}

```

Each edge requires three batch variables: look up the source node, look up the destination node, then add the edge between them. `NodeRef::var` creates a reference to a previously bound variable in the batch — this is how the edge step knows which two nodes to connect.

Chapter 13 — SurrealDB via REST HTTP

SurrealDB exposes a `/sql` endpoint that accepts multi-statement SurrealQL. The `surreal-http` backend is the simplest of the database backends: it builds SQL strings and posts them.

Bootstrap

```
async fn bootstrap(&self) -> Result<()> {
    if let Err(e) = self.run_sql(
        &format!("DEFINE NAMESPACE IF NOT EXISTS {}; ", self.ns),
        false, // no Surreal-Ns/Db headers
    ).await {
        tracing::warn!("SurrealDB bootstrap (DEFINE NAMESPACE): {e}");
    }
    if let Err(e) = self.run_sql(
        &format!("DEFINE DATABASE IF NOT EXISTS {}; ", self.db),
        true,
    ).await {
        tracing::warn!("SurrealDB bootstrap (DEFINE DATABASE): {e}");
    }
    Ok(())
}
```

SurrealDB's namespace definition must be executed at the *root level* without specifying a namespace or database in the HTTP headers. This is why `run_sql` has a `with_ns_db: bool` parameter.

Record IDs with angle brackets

SurrealDB record IDs must escape arbitrary string keys. The ID `talk:bythebay-20120727-monad-transformers-...` contains a colon, which SurrealDB would interpret as a nested record ID separator. The escape syntax uses Unicode Mathematical Angle Brackets (U+27E8 / U+27E9):

```
pub(super) fn surreal_id(table: &str, nid: &str) -> String {
    format!("{table}:\u{27E8}{nid}\u{27E9}")
}
```

Producing: `talk:{talk:bythebay-20120727-monad-transformers-...}`

The `\u{27E8}` escape inserts a Unicode scalar value directly into the string literal. Rust `char` is a Unicode scalar value, so any valid Unicode code point can appear this way.

Batch SurrealQL

```
async fn load_nodes_batch(&self, kind: &str, nodes: &NodeKindMap) ->
Result<BatchStats> {
    let table = kind.to_lowercase();
    let mut sql = String::new();
    for (nid, raw_props) in nodes {
        let id = surreal_id(&table, nid);
        let content = node_props(kind, nid, raw_props);
        let json = serde_json::to_string(&content).unwrap_or_else(|_|
            "{}".to_string());
    }
```

```

        sql.push_str(&format!("UPSERT {id} CONTENT {json};\n"));
    }
    self.run_batch(&sql).await
}

```

All UPSERT statements are concatenated into one SQL body and sent in a single HTTP request. SurrealDB executes them in order and returns an array of per-statement results:

```

async fn run_sql(&self, sql: &str, with_ns_db: bool) -> Result<BatchStats> {
    // ... build request, add basic auth, send ...
    let results: Vec<serde_json::Value> = resp.json().await?;
    let mut loaded = 0usize;
    let mut skipped = 0usize;
    for r in &results {
        if r.get("status").and_then(|s| s.as_str()) == Some("OK") {
            loaded += 1;
        } else {
            skipped += 1;
            tracing::warn!("{}", r["detail"].as_str().unwrap_or("?"));
        }
    }
    Ok(BatchStats { loaded, skipped })
}

```

Edge relationships are expressed with SurrealDB’s RELATE syntax:

```
RELATE talk:(talk:foo)->presented_at->event:(event:bar);
```

This reads naturally: *“talk:foo is related to event:bar via the presented_at edge type.”*

Chapter 14 — SurrealDB via the Rust SDK (WebSocket)

The surreal-sdk backend uses the official surrealdb Rust crate over a WebSocket connection. This demonstrates how a typed SDK compares to raw HTTP string building.

Generic type parameter on the connection

```

use surrealdb::{
    Surreal,
    engine::remote::ws::{Client as WsClient, Ws},
    opt::auth::Root,
    types::RecordId,
};

pub struct Loader {
    db: Surreal<WsClient>,
    concurrency: usize,
}

```

Surreal<WsClient> is a generic struct parameterized by the connection type. The type parameter WsClient encodes the protocol at the type level. Different protocol implementations (Ws, Http,

IndxDB) can be swapped without changing the query code — the `Surreal<T>` interface is the same for all of them.

Connection and authentication

```
pub async fn new(raw_url: &str, ns: &str, database: &str, user: &str, pass: &str,
concurrency: usize) -> Result<Self> {
    let ws_url = surreal_ws_address(raw_url);
    let db: Surreal<WsClient> = Surreal::new::<Ws>(ws_url.as_str())
        .await
        .map_err(|e| anyhow::anyhow!("SurrealDB connect ({ws_url}): {e}"))?;

    db.signin(Root { username: user.to_string(), password: pass.to_string() })
        .await
        .map_err(|e| anyhow::anyhow!("SurrealDB signin: {e}"))?;

    // Bootstrap at root level (before selecting ns/db):
    let _ = db.query(format!("DEFINE NAMESPACE IF NOT EXISTS {}; ", ns)).await;
    let _ = db.query(format!("USE NS {}; DEFINE DATABASE IF NOT EXISTS {}; ", ns,
database)).await;

    db.use_ns(ns).use_db(database).await
        .map_err(|e| anyhow::anyhow!("SurrealDB use_ns/use_db: {e}"))?;
    Ok(Self { db, concurrency })
}
```

The `Surreal::new::<Ws>(…)` call uses *turbofish* syntax to specify the type parameter `Ws` explicitly. Without it, Rust could not infer which protocol to use. The `::<Ws>` after the method name looks unusual but is required whenever type inference is ambiguous.

Typed upsert

```
let rid = RecordId::new(table.clone(), nid.clone());
let content = node_props(&kind, &nid, &raw_props);
db.upsert::<Option<serde_json::Value>>(rid)
    .content(content)
    .await
    .map(|_| ())
    .map_err(|e| anyhow::anyhow!("upsert {table} {nid}: {e}"))
```

`upsert::<Option<serde_json::Value>>(rid)` creates a builder that will return `Option<serde_json::Value>` as the response type. The turbofish `Option<serde_json::Value>` tells the SDK what to deserialize the response into. The response is discarded with `map(|_| ())` since we only need to know whether the operation succeeded.

Parameterized RELATE

```
db.query(format!("RELATE $from->{edge_table}->$to"))
    .bind(("from", from))
    .bind(("to", to))
    .await
```

Instead of interpolating node IDs directly into the query string (which would risk injection if IDs contained special characters), the SDK uses *bound parameters*. `$from` and `$to` are placeholders; `.bind(("from", from))` associates the `RecordId` value with the placeholder. The SDK serializes `RecordId` to the wire format automatically.

SDK vs HTTP: when to use each

Concern	HTTP (/sql)	SDK (WebSocket)
Setup complexity	Low	Medium
Connection cost	Per-request TCP handshake	Persistent WS connection
Type safety	None (strings)	High (typed records, bound params)
Batch sends	Yes (multi-statement body)	One call per operation
Injection risk	Manual escaping required	SDK handles escaping
Async	Yes	Yes

For simple scripts or quick experiments, the HTTP backend is easier. For production services with frequent writes, the SDK's persistent connection and type safety are worth the extra setup.

Part VI — Testing Strategy

Chapter 15 — Unit Tests with No Live Database

The twelve unit tests in `load.rs` test the query-generation functions without connecting to any database. This is possible because the Cypher and SurrealQL builders are pure functions that take Rust data and produce strings.

```
#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn test_cypher_str_escaping() {
        assert_eq!(falkor::cypher_str("hello"), "'hello'");
        assert!(falkor::cypher_str("it's").contains("\\'"));
        assert!(falkor::cypher_str("a\\b").contains("\\\\b"));
        assert!(falkor::cypher_str("a\\nb").contains("\\n"));
    }
}
```

`#[cfg(test)]` is a *conditional compilation attribute* — the `tests` module is compiled only when running `cargo test`. The module is inside `load.rs` and imports everything from the parent with `use super::*`. This gives the tests access to `pub(super)` items like `falkor::cypher_str` and `surreal_http::surreal_id`.

Testing BTreeMap ordering

```
#[test]
fn test_btreemap_deterministic_edge_order() {
    let mut edges: EdgeKindMap = BTreeMap::new();
    edges.insert(("talk:z".to_string(), "event:z".to_string()), None);
    edges.insert(("talk:a".to_string(), "event:a".to_string()), None);
    edges.insert(("talk:m".to_string(), "event:m".to_string()), None);

    let keys: Vec<_> = edges.keys().collect();
    assert_eq!(keys[0].0, "talk:a");
    assert_eq!(keys[1].0, "talk:m");
    assert_eq!(keys[2].0, "talk:z");
}
```

This test documents a design decision: “*we chose BTreeMap specifically for this property.*” Tests that document invariants are as valuable as tests that catch regressions.

Testing the SDK’s serialized wire format

```
#[test]
fn test_helix_batch_json_shape() {
    use helix_db::{DynamicQueryRequest, dsl::prelude::{PropertyValue, g,
    write_batch}};

    let req = DynamicQueryRequest::write(
        write_batch()
            .var_as("n0", g().add_n("Talk", vec![("nid",
    PropertyValue::from("talk:foo"))]))
            .var_as("n1", g().add_n("Talk", vec![("nid",
    PropertyValue::from("talk:bar"))]))
            .returning(["n0", "n1"]),
    );
    let json = serde_json::to_string(&req).expect("serialize
    DynamicQueryRequest");

    assert!(json.contains("\"write\""), "request_type is write: {json}");
    assert!(json.contains("\"queries\""), "queries field present: {json}");
    assert!(json.contains("\"returns\""), "returns field present: {json}");
}
```

This test exercises the real SDK and verifies that the JSON shape matches what HelixDB expects. It is not a mock — it uses the actual helix-db crate. Such tests are valuable because SDK wire formats can change between versions.

Chapter 16 — Integration Tests Against Live Backends

The integration tests in `tests/graph_integration_test.rs` verify that data was actually loaded correctly into the databases.

Graceful skipping

```
#[tokio::test]
async fn surreal_node_counts() {
    if !surreal::is_available().await {
        eprintln!("surreal not available – skipping");
        return;
    }
    assert_eq!(surreal::count_table("talk").await, EXPECTED_TALKS, "talk count");
    // ...
}
```

The test checks availability before making any assertions. This allows the full test suite to run in CI environments where only some backends are running, without marking tests as ignored. The test *passes* (returns without assertion failure) when the backend is unavailable — it is a pragmatic choice that trades false confidence for CI simplicity.

FalkorDB in an async test

```
#[tokio::test]
async fn falkor_node_counts() {
    if !tokio::task::spawn_blocking(falkor::is_available).await.unwrap() {
        eprintln!("falkor not available – skipping");
        return;
    }
    assert_eq!(
        tokio::task::spawn_blocking(||
falkor::count_label("Talk")).await.unwrap(),
        EXPECTED_TALKS, "talk count"
    );
}
```

`falkor::is_available` and `falkor::count_label` are synchronous functions (they call the Redis crate). Inside a `#[tokio::test]` async test they must be wrapped in `spawn_blocking`. Note the different closure forms:

- `spawn_blocking(falkor::is_available)` — passes a *function pointer* directly, no closure needed because the function takes no arguments.
- `spawn_blocking(|| falkor::count_label("Talk"))` — requires a closure to capture the string argument.

Graph traversal queries

The SurrealDB integration tests demonstrate nested subquery traversal:

```
pub async fn groups_of_speaker(speaker_nid: &str) -> HashSet<String> {
    let q = format!(
        "SELECT out.nid AS nid FROM part_of \
        WHERE in IN \
        (SELECT VALUE out FROM presented_at \
        WHERE in IN \
        (SELECT VALUE in FROM presented_by WHERE out = {}));",
    );
```

```

        sid("speaker", speaker_nid)
    );
    sql(&q).await[0]["result"]
        .as_array()
        .unwrap_or(&vec![])
        .iter()
        .filter_map(|r| r["nid"].as_str().map(str::to_string))
        .collect()
}

```

This traverses three hops: *speaker* → *talks* → *events* → *groups*. The innermost query finds talk IDs for the speaker; the middle query finds event IDs for those talks; the outer query finds group IDs for those events. The result is deduplicated by the `HashSet<String> collect`.

The same traversal in HelixDB’s SDK style:

```

pub async fn groups_of_speaker(speaker_nid: &str) -> HashSet<String> {
    let v = sdk_read(DynamicQueryRequest::read(
        read_batch()
            .var_as("speaker", g().n_with_label_where("Speaker",
SourcePredicate::eq("nid", speaker_nid)))
            .var_as("groups",
                g().n(NodeRef::var("speaker"))
                    .in_(Some("PRESENTED_BY")) // walk backward along
PRESENTED_BY edges
                    .out(Some("PRESENTED_AT")) // forward along PRESENTED_AT
edges
                    .out(Some("PART_OF")) // forward along PART_OF edges
                    .dedup()
                    .value_map(Some(vec!["nid"])))
            .returning(["groups"]),
    )).await;
    // extract nid values from the result ...
}

```

The HelixDB version expresses the same traversal as a linear chain of graph steps: start at a node, walk edges, deduplicate, return properties. The `.in_(Some("PRESENTED_BY"))` step means “*traverse incoming edges labelled PRESENTED_BY*” — i.e., go from Speaker to the Talks that referenced it.

And in Cypher (FalkorDB):

```

MATCH (sp:Speaker {nid:'speaker:adam-warski'})
  <-[:PRESENTED_BY]-(:Talk)
  -[:PRESENTED_AT]->(:Event)
  -[:PART_OF]->(g:Group)
RETURN DISTINCT g.nid

```

All three express the same graph traversal pattern; each language has its own idiom for it.

Part VII — Rust Language Deep Dives

Chapter 17 — Error Handling with anyhow

The codebase uses `anyhow::Result<T>` everywhere as the function return type for fallible operations.

The `?` operator

```
pub async fn save_talk(record: &TalkRecord, output_dir: &Path) -> Result<PathBuf>
{
    let dir = output_dir.join("talks");
    fs::create_dir_all(&dir).await?;
    let path = dir.join(format!("{}", record.id));
    let json = serde_json::to_string_pretty(record)?;
    fs::write(&path, json).await?;
    Ok(path)
}
```

Each `?` after a `Result`-returning expression does three things:

1. If the result is `Ok(v)`, unwrap to `v` and continue.
2. If the result is `Err(e)`, convert `e` to the function's error type using `From::from`.
3. Return the converted error from the function.

Because the function returns `anyhow::Result`, the `From impl` accepts any error type that implements `std::error::Error`, providing seamless error propagation.

Adding context with `.context()`

```
let raw = std::fs::read_to_string(&path)
    .with_context(|| format!("read {}", path.display()))?;
```

`with_context` wraps the error with a human-readable message. When this error eventually reaches `main` and is printed, the output shows a chain:

```
Error: read data/talks/broken.json
Caused by: No such file or directory (os error 2)
```

The `||` makes `with_context` take a closure rather than a string so that the format string is only evaluated when an error actually occurs.

`bail!` for early-exit errors

```
if !is_safe_label(kind) {
    bail!("unsafe label: {kind}");
}
```

`bail!` is a macro that is equivalent to `return Err(anyhow::anyhow!(...))`. It produces a clean one-liner for validation guards.

Chapter 18 — Ownership, Borrowing, and Cloning

Why `clone()` appears in async spawns

```
for (nid, props) in nodes {
    let nid = nid.clone();           // clone because `nodes` is borrowed
    let raw_props = raw_props.clone();
    let db = self.db.clone();        // Arc clone – cheap
    let sem = sem.clone();           // Arc clone – cheap
    handles.push(tokio::spawn(async move {
        // uses nid, raw_props, db, sem
    }));
}
```

`tokio::spawn` requires that the future is 'static — it must not borrow any stack variables. Since `nodes` is a reference to data owned outside the loop, `nid` and `raw_props` (which are references into `nodes`) cannot be moved into the spawn. Cloning creates owned copies that the task can move.

`db.clone()` is cheap because `Surreal<WsClient>` internally wraps its state in an `Arc`, so `clone` just increments the reference count. This is the *interior mutability + reference counting* pattern: the connection is shared, not copied.

Arc vs Rc

`Arc<T>` is an atomic reference-counted pointer: safe to clone from multiple threads. `Rc<T>` is a single-threaded reference-counted pointer: faster but not thread-safe. Tokio's async runtime may run tasks on any thread in the pool, so `Arc` is required. The Rust compiler enforces this: using `Rc` inside a `tokio::spawn` is a compile error because `Rc<T>` does not implement `Send`.

Chapter 19 — Closures and Iterator Adapters

Rust's iterators are lazy: they compute values on demand. This enables chains of transformations with zero intermediate allocations.

The slugify chain revisited

```
text.to_lowercase()           // String: allocates
    .chars()                   // iterator: no allocation
    .map(|c| ...)               // iterator: no allocation
    .collect::<String>()        // String: allocates
    .split('-')                 // iterator: no allocation
    .filter(|s| !s.is_empty()) // iterator: no allocation
    .map(str::to_string)        // iterator: no allocation
    .collect()                  // Vec<String>: allocates
```

Only two allocations occur: the lowercase `String` and the final `Vec`. Without iterators, a naive implementation might allocate a dozen times.

`flat_map` for one-to-many transformations

```
let stage2: Vec<&str> = stage1
    .into_iter()
```

```
.flat_map(|s| s.split(';'))
.collect();
```

`flat_map` is equivalent to `map` followed by `flatten`. `s.split(';')` produces an iterator of `&str` sub-slices; `flat_map` flattens these into one stream. This is the idiomatic way to split nested structures.

Method references as closures

```
.filter_map(|r| r["nid"].as_str().map(str::to_string))
```

`str::to_string` is a *method reference* — a pointer to the `str::to_string` method — used as a closure of type `fn(&str) -> String`. Method references can be used anywhere a closure of matching type is expected, without the syntax overhead of a lambda.

Chapter 20 — Lifetimes

Lifetimes are Rust’s way of tracking how long borrows are valid.

Explicit lifetime in `str_prop`

```
fn str_prop<'a>(props: &'a serde_json::Value, key: &str) -> &'a str {
    props.get(key).and_then(|v| v.as_str()).unwrap_or("")
}
```

The lifetime annotation `'a` says: “*the returned `&str` lives as long as the `props` reference*.” Without this annotation, the compiler cannot determine whether the returned `&str` borrows from `props` or from `key` or is a static string. With it, the caller knows the returned slice is valid as long as `props` is valid.

The empty string `""` in `unwrap_or("")` has lifetime `'static` (string literals live forever), which is compatible with any lifetime `'a` via covariance.

Lifetime elision

Most functions in the codebase do not have explicit lifetime annotations because Rust infers them automatically via *lifetime elision rules*. The rules cover the common cases:

1. Each reference parameter gets its own lifetime.
2. If there is exactly one reference input, the output lifetime is the same.
3. If one of the inputs is `&self` or `&mut self`, the output lifetime is `'self`.

`str_prop` falls outside rule 2 because it has two reference parameters, so the lifetime must be stated explicitly.

Chapter 21 — Pattern Matching

Rust’s match expressions are exhaustive — the compiler verifies that every possible case is handled.

Matching on `redis::Value`

```
fn scalar_int(val: &redis::Value) -> usize {
    let redis::Value::Array(outer) = val else { return 0 };
    let rows = match outer.get(1) {
        Some(redis::Value::Array(r)) => r,
        _ => return 0,
    };
    let row = match rows.first() {
        Some(redis::Value::Array(r)) => r,
        _ => return 0,
    };
    let cell = row.first().unwrap_or(&redis::Value::Nil);
    match cell {
        redis::Value::Int(n) => *n as usize,
        redis::Value::Array(pair) => match pair.get(1) {
            Some(redis::Value::Int(n)) => *n as usize,
            _ => 0,
        },
        _ => 0,
    }
}
```

The `let X = val else { return 0 }` construct is a *let-else statement*: destructure `val` into the pattern `redis::Value::Array(outer)`, or execute the `else` block if it does not match. This is cleaner than a nested `match` when the non-matching case is a simple early return.

The nested `match cell` pattern deals with the fact that FalkorDB may represent integer results as either a bare `Int` or as a `[type, value]` pair. This kind of defensive multi-case matching is common when working with external protocols that have varying response formats.

Matching tuples in `parse_speaker_string`

```
let split_and = match (comma_pos, raw.find(" and ")) {
    (Some(c), Some(a)) => a < c,
    (None, _) => true,
    _ => true,
};
```

Matching on a tuple `(Option<usize>, Option<usize>)` allows all four combinations to be expressed concisely. The wildcard `_` in `(None, _)` ignores the second field; the final `_ => true` catches all remaining cases (here: `(Some(_), None)`).

Chapter 22 — Important Design Decisions

1. One binary, five backends — not five binaries

All five backends live in `src/bin/load.rs` as inline modules. This has several advantages:

- A single `cargo run --bin load` command; no need to remember five binary names.
- Shared code (`collect`, `node_props`, `GraphLoader`, helper functions) is not duplicated.

- Unit tests in one `#[cfg(test)]` block cover all backends.

The trade-off is a larger binary and slightly longer compile times. At this scale that is negligible.

2. Deduplication by first-seen wins

When the same node ID appears in multiple talk files, the first-seen properties are kept:

```
.entry(node.id)
.or_insert(node.properties);
```

This is correct for nodes like `Event` and `Group` whose properties are stable across files. For `Speaker` nodes it means the first biography and company encountered wins — a pragmatic choice that avoids needing a merge strategy.

3. BTreeMap over HashMap for determinism

Every data structure that holds nodes or edges is a `BTreeMap`. The cost is $O(\log n)$ operations instead of $O(1)$. The benefit is reproducible test output and stable loading order. Graph databases that use node insertion order for query planning benefit from predictable order.

4. The trait bootstrap hook

Adding `bootstrap()` to the trait — with a default no-op — allows each backend to perform one-time setup without the caller needing to know which backends require it. The alternative (calling `bootstrap` explicitly in `main`) would expose backend-specific knowledge in the orchestrating code.

5. Structured properties from a single function

`node_props` is the single source of truth for what the graph databases see. It enforces field names (including renames like `abstract_text`) and provides defaults for missing values. All backends call this function; none duplicates it.

6. Tests that test the testing infrastructure

The integration test helpers (`surreal::talks_by_speaker`, `helix::groups_of_speaker`, etc.) are themselves non-trivial SurrealQL and HelixDB SDK queries. A failure in a test helper produces a misleading test failure. The fix is to keep helpers simple, document their expected behaviour, and use them in multiple tests so that a helper regression becomes visible quickly.

Chapter 23 — Key Rust Crates Used

anyhow — ergonomic error handling

`anyhow::Result<T>` is `std::result::Result<T, anyhow::Error>`. `anyhow::Error` can wrap any error type that implements `std::error::Error`. The crate provides the `?` operator integration, `context`, `with_context`, `anyhow!`, and `bail!`. It is the go-to choice for application code where you want errors to propagate without defining custom error enums.

tokio — async runtime

Tokio provides the executor that runs futures, the I/O primitives (`tokio::fs`, `tokio::net`), synchronization (`Mutex`, `Semaphore`, `OnceLock`), and task management (`tokio::spawn`, `block_in_place`, `spawn_blocking`). The full feature flag enables all subsystems.

request — HTTP client

`request` is built on `hyper` and provides an ergonomic async HTTP client. Key patterns used in this codebase: - `.json(&body)` — serializes a `Serialize` value as the request body. - `.json::<T>().await` — deserializes the response body into a `T`. - `.basic_auth(user, Some(pass))` — adds HTTP Basic Auth. - `.header(k, v)` — adds custom headers. - `Client::builder().timeout(...).build()` — configures a shared client.

serde and serde_json

`serde` is Rust's serialization framework. The `#[derive(Serialize, Deserialize)]` macros generate efficient, zero-allocation serialization code at compile time. `serde_json` provides JSON support and the dynamic `Value` type.

clap — command-line argument parsing

The `derive` feature lets you define the CLI interface as a Rust struct with attributes. The parser is generated at compile time with full help text, type conversion, validation, and shell completions.

redis — Redis (and FalkorDB) client

A synchronous Redis client that supports arbitrary commands, including FalkorDB's `GRAPH.QUERY` extension.

helix-db — HelixDB Rust SDK

Provides `Client`, `DynamicQueryRequest`, and the `dsl::prelude` items: `g()`, `read_batch()`, `write_batch()`, `NodeRef`, `SourcePredicate`, `PropertyValue`. The DSL compiles graph traversals to a wire format that HelixDB understands.

surrealdb — SurrealDB Rust SDK

Provides `Surreal<T>`, `RecordId`, and protocol engine types. The SDK uses Rust's type system to enforce correct usage: you cannot run a query before selecting a namespace and database.

Part VIII — Putting It All Together

Chapter 24 — Reading the Data Pipeline End to End

Let us trace one talk from raw HTML to graph database query.

Step 1 — Scraping

The scraper fetches the event page for an SF Scala meetup. The description contains:

```
(1) Monad Transformers in Scalamachine and Scaliak
Jordan West, StackMob
...abstract text...
```

The `talks_numbered_sections` strategy fires: it sees (1) as a numbered section header, extracts "Monad Transformers in Scalamachine and Scaliak" as the title, then scans the following lines for `speaker_from_prose`. "Jordan West, StackMob" matches `looks_like_speaker_field` (has a comma, short, first char uppercase), and `parse_speaker_string` produces `SpeakerData { name: "Jordan West", company: Some("StackMob"), bio: None }`.

Step 2 — Building the graph record

`store::build_talk_record` receives the `TalkData` and `EventData`. It slugifies names, constructs node and edge IDs, and returns a `TalkRecord`:

```
let talk_nid    = "talk:bythebay-20120727-monad-transformers-...-jordan-west";
let event_nid   = "event:sf-scala-69910422";
let group_nid   = "group:sf-scala";
let speaker_nid = "speaker:jordan-west";
```

Three edges are created: `PRESENTED_AT`, `PART_OF`, `PRESENTED_BY`.

Step 3 — Loading

`collect()` reads 163 talk files. The `speaker:jordan-west` node will appear in exactly one file (Jordan West gave one recorded talk). The loader calls `load_nodes_batch("Speaker", ...)` which delegates to the selected backend.

For SurrealDB HTTP, this emits:

```
UPSERT speaker:{speaker:jordan-west} CONTENT
  {"nid":"speaker:jordan-west","name":"Jordan
West","bio":"","company":"StackMob"};
```

For HelixDB SDK, this becomes part of a `write_batch()` call:

```
batch.var_as("n42", g().add_n("Speaker", vec![
  ("nid",    PropertyValue::from("speaker:jordan-west")),
  ("name",   PropertyValue::from("Jordan West")),
  ("bio",    PropertyValue::from("")),
  ("company", PropertyValue::from("StackMob")),
]))
```

For FalkorDB, it is part of an `UNWIND` array:

```
{nid:'speaker:jordan-west', props:{name:'Jordan West', bio:'',
company:'StackMob'}}
```

Step 4 — Querying

The integration test verifies that Stefan Webb (not Jordan West, since Jordan has only one recorded talk) shows up in four talks across the dataset:

```
let talks = helix::talks_by_speaker("speaker:stefan-webb").await;
assert_eq!(talks.len(), 4);
```

The HelixDB query traverses the graph:

```
g().n_with_label_where("Speaker", SourcePredicate::eq("nid", "speaker:stefan-
webb"))
.in_(Some("PRESENTED_BY"))
.value_map(Some(vec!["nid"]))
```

Starting from the Stefan Webb Speaker node, follow incoming PRESENTED_BY edges to reach all Talk nodes that reference him, then return their nid properties.

Chapter 25 — Summary and Further Reading

What you have seen

This codebase is a compact but complete Rust application demonstrating:

Concept	Where
Trait with async methods + default impl	GraphLoader trait
impl Trait monomorphization	load_graph(&impl GraphLoader, ...)
OnceLock for lazy static regex	parse.rs fn re_*() functions
macro_rules! for control flow	try_strategy! in parse.rs
block_in_place for sync in async	FalkorDB load_nodes_batch
spawn_blocking in tests	FalkorDB integration tests
Arc + Semaphore for rate limiting	all async backends
BTreeMap for deterministic order	collect() in load.rs
pub(super) visibility	Cypher builders in mod falkor
Builder pattern	write_batch().var_as(...).returning(...)
Turbofish type annotation	Surreal::new:::<Ws>(...)
Lifetime annotations	fn str_prop<'a>(...) -> &'a str
Graceful if !available { return } tests	integration tests
#[serde(rename)] + skip_serializing_if	models.rs
anyhow::bail! + .with_context()	throughout

Further reading

- *The Rust Programming Language* (Brown University edition) — the canonical free book, recently updated to cover async.
- *Rust for Rustaceans* by Jon Gjengset — advanced type system and lifetime topics.
- *Programming Rust* (O'Reilly, 2nd ed.) — comprehensive reference.
- The tokio tutorial at tokio.rs — async in depth.
- The serde documentation at serde.rs — serialization internals.
- The clap derive reference — all attribute options.

End of book.
